

开源项目学习 · 小白指南

Qdrant 向量数据库

让机器按"意思"找东西 —— 从 0 到搭一个迷你语义电影搜索引擎

项目

github.com/qdrant/qdrant

版本

v1.19.1 (Rust)

日期

2026-10-01

实验

全部命令已在本机验证

这份指南回答的 12 个疑问 疑问即目录

01 Qdrant 是什么? "向量数据库"和 MySQL 有什么本质区别?

02 "向量 / embedding"到底是什么? 文字怎么变成一串数字?

03 为什么需要它? 没有向量数据库之前, "找相似"怎么做?

04 余弦相似度是什么? 凭什么一个分数能衡量"像不像"?

05 几千万条数据一条条比, 不慢吗? Qdrant 怎么做到毫秒返回?

06 核心概念: Collection、Point、Payload 是什么? 和传统数据库怎么对应?

07 真实的文字怎么变向量? embedding 模型从哪来?

08 RAG 是什么? Qdrant 在大模型应用里站在哪个位置?

09 "过滤搜索"是什么? 为什么 Qdrant 特别强调 filtering?

10 数据量大到内存装不下怎么办? 什么是量化?

11 Qdrant、FAISS、Milvus、pgvector、Chroma.....到底怎么选?

12 这个项目给我什么? 我能用它做出什么?

01 Qdrant 是什么？"向量数据库"和 MySQL 有什么本质区别？

一句话：Qdrant 是一个专门存放"意义"、并按"意义相似"来检索的数据库。 它读作 *quadrant* (象限)，用 Rust 写成，既提供带 API 的网络服务，也在 2024 年后成为向量数据库领域口碑最好的开源项目之一。

MySQL 按"精确匹配"检索，Qdrant 按"语义距离"检索——这是两种完全不同的提问方式：

- 问 MySQL: "title 字段等于《星际穿越》的行是哪几行？" ——答案非对即错，没有"差不多"。
- 问 Qdrant: "哪些内容最接近 '在太空深处寻找人类的未来'？" ——答案是一张按相似度排序的榜单。



图 1-1 同一个提问动机，两种完全不同的检索哲学

它同时保留了传统数据库的本领：每个点可以带 **payload (元数据)** 并做**条件过滤**——所以 Qdrant 自我介绍时强调 "vector **database**" 而不只是 "vector search library" (对比第 11 问的 FAISS)。

02 Q2 | "向量 / embedding"到底是什么？文字怎么变成一串数字？

术语卡 **向量 (vector)**：一串固定长度的数字，如 `[0.12, -0.87, ...]`。**embedding (嵌入)**：把一段内容（文字/图片/音频）"压"成这样一个向量的过程和结果。

关键直觉：**embedding 模型把"意思相近的文本"映射到高维空间里相邻的位置。**这不是把字编码成编号——"猫吃鱼"和"小猫捕食鱼类"几乎没有共同字符，但它们的向量夹角极小；而"猫吃鱼"和"今天股市大涨"的向量几乎垂直。



图 2-1 embedding：把"意思"变成高维空间中的"位置"

为什么一串数字能代表"意思"？可以类比坐标：地图上两个点的(经度, 纬度)近，两地就真的近。embedding 模型把每段文本放到一个 512 维（本实验用的 bge-small-zh 是 512 维）的"意义地图"上——**训练时模型见过海量文本，学会了把"同义""相关"的内容放到邻近坐标。**维度本身没有人工含义，是模型从数据里学出来的压缩坐标。

记忆锚点：**embedding = 意义的坐标；向量数据库 = 按坐标距离找邻居的仓库。**

03 Q3 | 为什么需要它？没有向量数据库之前，“找相似”怎么做？

人类的“找相似”技术一共进化了四代：

世代	做法	致命弱点
① 关键词 / LIKE	字符串匹配，如数据库 <code>LIKE '%猫%'</code>	换个说法就找不到：“猫吃鱼”搜不到“小猫捕食鱼类”
② 倒排索引 (Elasticsearch / BM25)	把文档拆成词，建“词 → 文档列表”的字典	仍然基于 字面 ，同义词、换说法依然失配
③ 自己算向量自己比	把所有向量放内存/CSV，逐条算距离取最小	数据一多就崩溃：百万条 × 512 维的暴力比对既慢又占内存，还没有增删改、过滤、持久化
④ 向量数据库 (Qdrant 等)	专职存储 + 索引 (HNSW) + 过滤 + 增删改 + 服务化 API	——（这就是本指南的主角）



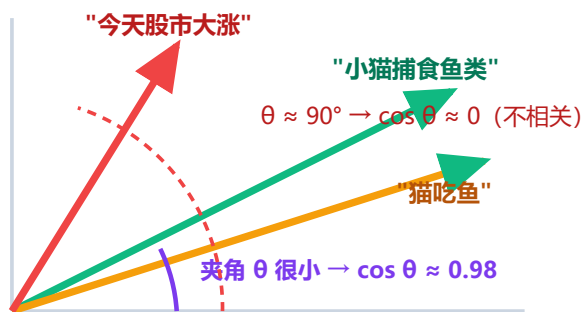
图 3-1 “找相似”的四代进化：从字面到语义

本指南实验 2 会让你亲眼看到第①代和第④代的差距：三个查询用关键词搜索全部 **0 条命中**，Qdrant 语义搜索全部命中正确答案（见实验步骤 4 的 预期结果）。

04 Q4 | 余弦相似度是什么？凭什么一个分数能衡量"像不像"？

术语卡 余弦相似度 (Cosine Similarity)：用两个向量**夹角**的余弦值衡量方向是否一致，取值 $[-1, 1]$ ，越大越相似。

直觉：**只看方向，不看长短**。两段文本哪怕一个长一个短，只要"意思指向"同一个方向，夹角就小、分数就高。



本机实验实测 (查询"猫吃鱼")

猫吃鱼	1.0000	← 自己 vs 自己
小猫捕食鱼类	0.9838	← 换说法仍很高
今天股市大涨	0.0000	← 无关=0

$$\text{公式: } \cos \theta = (A \cdot B) / (|A| \times |B|)$$

图 4-1 余弦相似度 = 比"方向"不比"长度"; 右框为本机真实运行结果

除了 Cosine, Qdrant 还内置 **欧氏距离 (Euclid, 比"直线距离")** 和 **点积 (Dot, 推荐系统常用)**，建集合时二选一。**文本 embedding 的业界默认是 Cosine。**

05 Q5 | 几千万条数据一条条比，不慢吗？Qdrant 怎么做到毫秒返回？

暴力比对 (exact search) 确实太慢。Qdrant 默认使用 **HNSW (分层可导航小世界图) 近似最近邻索引**——用“可能错过亿点点、但快几百倍”换速度的技术，思想类似“先走高速，再走省道，最后走小巷”：

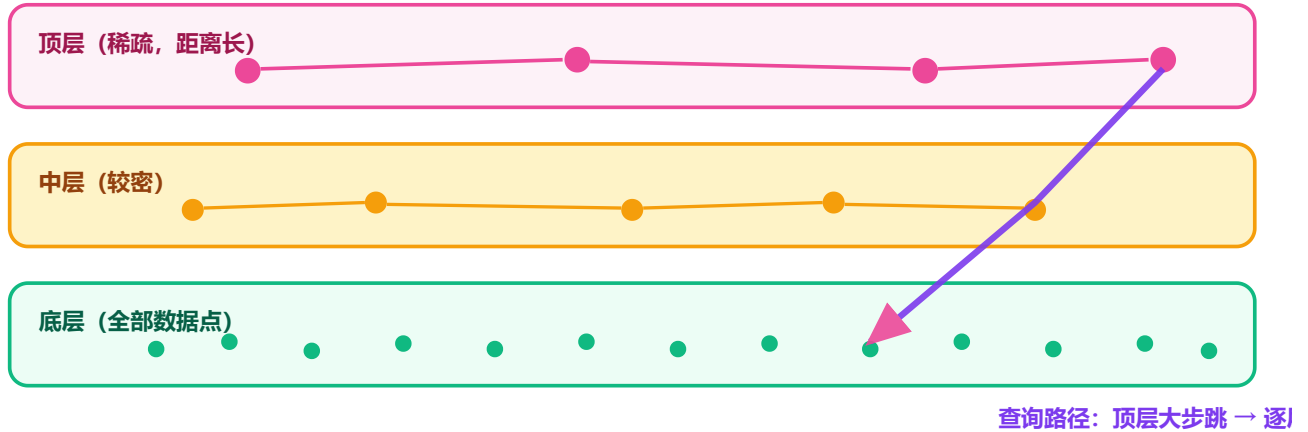


图 5-1 HNSW: 每层是一张“邻居图”，查找时自顶向下逐层逼近目标

要点三个：

- **快**：几千万点也只需访问几十个节点（对数级复杂度），毫秒返回；
- **"近似"的含义**：结果以 `hnsw_ef` 等参数权衡精度与速度——99% 的场景你感知不到差异；
- **建索引要时间**：写入后 Qdrant 在后台异步建图，点数少时（如本实验 16 条）甚至直接暴力比对，小数据本来就快。

Qdrant 的概念体系非常克制，只有四层：

Qdrant	类比 MySQL	是什么
Collection 集合	database / table	一批点的容器，创建时声明 向量维度 + 距离度量
Point 点	一行记录	<code>(id, vector, payload)</code> 三元组——检索的基本单位
Payload 载荷	行的其他列	点附带的任意 JSON 元数据（标题、年份、来源.....）
Distance 距离度量	—（MySQL 没有）	建集合时定死：Cosine / Euclid / Dot

Collection [movies]

Point id=1

vector: [0.03, -0.11, ...]
title: "流浪地球2"
genre: "科幻" year: 2023

Point id=2

vector: [0.02, -0.09, ...]
title: "星际穿越"
genre: "科幻" year: 2014

Point id=16

vector: [0.44, 0.05, ...]
title: "让子弹飞"
genre: "剧情" year: 2010

每个 Point = id（唯一）+ vector（检索用）+ payload（过滤/展示用）

图 6-1 Collection / Point / Payload 全家福——本实验 movies 集合的真实结构

为什么 vector 和 payload 分开？ 分工明确：**vector 负责“像不像”，payload 负责“是不是”**。向量走 HNSW 索引做语义近邻，payload 走普通索引做精确条件过滤——两者在查询里组合使用（见第 9 问）。

07 Q7 | 真实的文字怎么变向量? embedding 模型从哪来?

第 2 问说了"向量是意义的坐标", 但坐标**由谁颁发**? 答案是 embedding 模型。本实验用的是 **BGE-small-zh-v1.5** (北京智源发布、对中文优化、512 维、模型仅约 100MB), 通过 **fastembed** 这个库在本地 CPU 上运行:



图 7-1 数据 → embedding 模型 → Qdrant 的分工。模型与数据库是解耦的

三个工程上必须知道的规则:

- **入库和查询必须用同一个模型**——不同模型的空间坐标互不相通, 混用等于拿北京坐标在纽约找路;
- **维度由模型定死**——bge-small-zh 是 512 维, 建 Collection 时 `size` 必须一致;
- **模型在哪跑都行**——本地 (fastembed / ollama 的 `nomic-embed-text`) 或云端 API (OpenAI / 智谱), Qdrant 无感知。

⚠ 本机坑: 计划用 ollama 拉 `nomic-embed-text` 时被 Clash fake-IP 劫持 DNS, ollama 拒绝重定向到 198.18.x.x 非公网地址。改用 fastembed (走 HTTPS_PROXY 正常下载)。你日后若复现, 二选一即可。

08 Q8 | RAG 是什么？Qdrant 在大模型应用里站在哪个位置？

术语卡 **RAG (Retrieval-Augmented Generation, 检索增强生成)**：回答问题前先去知识库里**检索**相关资料，把资料塞进提示词再让大模型**生成**答案。

你已经在学的 langchain / langgraph 项目里，RAG 是头号应用模式，而 **Qdrant 就是 RAG 链路里的“外挂记忆体”**——大模型不知道你的私有文档，但可以从 Qdrant 里现查现用：

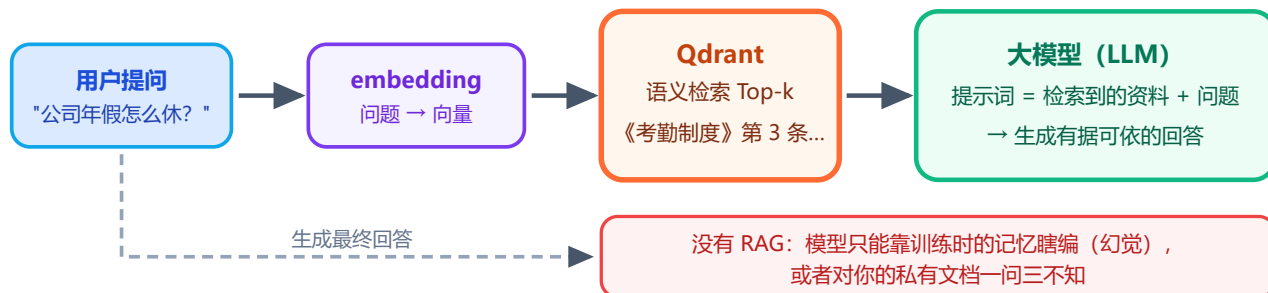


图 8-1 RAG 五步：提问 → 向量化 → Qdrant 检索 → 拼进提示词 → LLM 生成

一条产业链的位置感：**ollama 跑模型（你已学）** → **langchain 编排流程（你在学）** → **Qdrant 存知识（本项目）**——三者拼起来就是一个完整的私有知识库问答系统。Qdrant 官方甚至提供 `langchain-qdrant` 集成包，后续做 langchain RAG 实验时可以直接接入今天建的 movies 集合。

09 Q9 | "过滤搜索"是什么? 为什么 Qdrant 特别强调 filtering?

真实需求几乎总是："在符合条件的那部分里，找最像的。" 比如"在 2015 年后的科幻片里找和这段简介像的"。这就是过滤搜索——payload 条件（第 6 问）和向量近邻（第 5 问）联合作战。

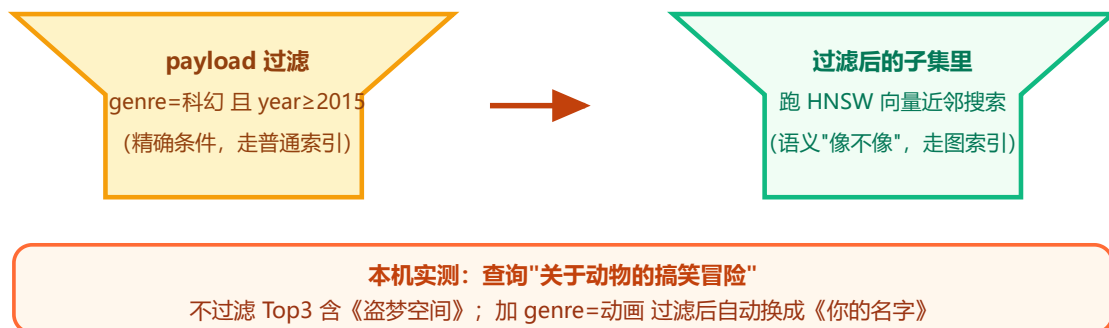


图 9-1 过滤搜索 = 先按条件收窄子集，再在子集内做语义搜索（本机实验 3 实测）

Qdrant 强调 **extended filtering**，因为它把过滤做在了索引层（称为 **filterable HNSW**）：**是在图里"只走满足条件的路"**，而不是先搜 Top-100 再把不合格的扔掉。后者在条件命中稀少时会全军覆没——先搜后滤可能一条都不剩，先滤后搜永远有答案。支持 `must / should / must_not` 组合，等值、范围、全文匹配都可以。

10 Q10 | 数据量大到内存装不下怎么办？什么是量化？

算一笔账：1 亿条 × 512 维 × 4 字节 (float32) ≈ **200 GB**——全放内存很奢侈。Qdrant 的答案有两层：

① 量化 (Quantization) ——把每个数变“矮”：

方式	存储	压缩比	说明
原始 float32	4 字节/维	1×	默认
Scalar 量化	1 字节/维 (int8)	~4×	精度损失极小
Product 量化	每段向量用码本编号	最高 ~64×	损失较大，适合超大库
二值量化	1 bit/维	~32×	配合 oversampling 使用



图 10-1 标量量化：把“高精度小数”压成“整数刻度”，体积÷4

② 分级存储——热的放内存，冷的放磁盘：Qdrant 支持把原始向量留在磁盘 (mmap)，内存只放量化后的向量和索引，搜索时用量化向量粗筛、偶尔回磁盘校验原值 (rescoring)。单机扛亿级向量因此成为可能。

一句话：量化省内存，分级存储省钱，HNSW 省时间——三者合体就是“亿级向量毫秒级搜索”。

这些名字极易混淆，用两个维度切开就清楚了：它是“库”还是“服务”？它是“专职”还是“兼职”？

名字	形态	一句话定位	适合
Qdrant	服务 (Rust)	过滤能力最强的专职向量数据库	生产 RAG / 语义搜索，单机到分布式
FAISS	库 (C++/Python)	Meta 出品的索引算法集，无服务无增删改便利	研究、离线批处理、嵌入自家系统
Milvus	服务 (Go/C++)	重装 distributed 风格，组件多	超大规模团队，有专职运维
pgvector	PG 插件	给 PostgreSQL 加向量能力	数据已在 PG、想少一个组件时
Chroma	嵌入式 (Python)	一行代码起步的原型玩具	notebook 快速试验、demo
Elasticsearch	服务 (Java)	搜索老牌巨头后加的向量功能	已用 ES 全文检索想顺带向量

初学者的务实路径：原型期用 Chroma（零部署）→ 需要真服务时上 Qdrant（单二进制 30MB 就能跑，本实验已验证）→ 数据已在 PG 就先试 pgvector。**Qdrant 对“过滤 + 向量”复合查询的支持是其最常被选中的理由**（第 9 问的 filterable HNSW）。

另外它还是少数把 **Rust 内存安全 + 单二进制部署 + Web UI + 三种 API (REST/gRPC/客户端)** 同时做齐的向量数据库——本实验用的是官方发布的 Windows 原生二进制，连 Docker 都没装。

12 Q12 | 这个项目给我什么？我能用它做出什么？

能力清单（学完本指南你已实际拥有）：

本地向量数据库服务

中文 embedding 编码 (fastembed + BGE)

建集合 / 写入 / 语义搜索

payload 条件过滤搜索

REST API 直玩

结合你的水平（已在学 ollama / langchain / langgraph / crewai），6 个马上能做的项目：

#	项目	怎么用 Qdrant
1	私人文档问答 (RAG)	把 PDF/笔记切片入库，langchain 检索 + ollama 生成——你的第一个私有知识库
2	"以文搜图"相册	CLIP 模型把照片编码入库，用一句话搜出"海边日落的照片"
3	推荐系统	把用户看过的内容向量求均值作为用户画像，搜最近邻即"猜你喜欢"
4	去重/聚类工具	新闻、批改作业时找语义重复项：向量搜索 score 超阈值即判定重复
5	Agent 长期记忆	给 crewai/langgraph 的智能体加"回忆"：对话历史入库，行动前检索相关记忆
6	混合搜索 (hybrid)	向量语义 + BM25 关键词双路召回再融合，Qdrant 1.10+ 原生支持，效果更强

动手实验：迷你语义电影搜索引擎 全部命令已实际执行验证 · 2026-10-01

环境备忘（本机已就绪，勿重复安装）： Qdrant v1.19.1 Windows 原生二进制（免 Docker）；Python 3.14 + qdrant-client 1.19.1 + fastembed 0.8.1；模型 BAAI/bge-small-zh-v1.5 已缓存。两个坑：① Clash 代理会劫持 localhost 请求（已 setx NO_PROXY）；② Python 务必 UTF-8 模式（PYTHONUTF8=1 已设）。

STEP 1 启动 Qdrant 服务器，看它“活着”

怎么操作

PowerShell 进入项目目录： `cd`
`F:\reminder\vector_database_qdrant` 后台运行
`.\qdrant.exe` (或 `Start-Process`
`.\qdrant.exe`)

目的

把向量数据库作为本地服务跑起来，监听
6333(REST)/6334(gRPC)

实现了什么

一个真·数据库服务在你机器上待命，数据会持久化到
`storage\` 目录

怎么验证

浏览器或 PowerShell 访问 `http://localhost:6333`

✅ 预期结果 (实测) 返回 JSON:

```
{"title": "qdrant - vector search engine", "version": "1.19.1", "commit": "6ab21cac..."}
```

STEP 2 安装 Python 客户端 + embedding 库

怎么操作

```
pip install "qdrant-client[fastembed]"
```

目的

qdrant-client 负责和数据库对话，fastembed 负责把中文变向量 (本地 CPU 推理，不需要任何 API key)

实现了什么

获得“文本 → 512 维向量 → 存库 → 搜索”的完整工具链

怎么验证

```
pip show qdrant-client fastembed
```

 能看到版本号

✅ 预期结果 (实测) `qdrant-client 1.19.1 / fastembed 0.8.1`。首次运行 embedding 时会自动下载约 100MB 的 `bge-small-zh-v1.5` 模型到本地缓存，之后离线可用。

STEP 3 入库：16 部电影简介 → 向量 → Qdrant

怎么操作

```
cd
F:\reminder\vector_database_qdrant\exercise
后运行 python exp1_ingest.py
```

目的

走完"文本 → embedding → 建集合 → upsert"全流程
(脚本四步都有注释)

实现了什么

建出 `movies` 集合：16 个 Point，每个含 512 维向量
+ payload (title/genre/year/desc)

怎么验证

看脚本最后打印的集合状态

✅ 预期结果 (实测输出)

已编码 16 条文本，每条 512 维
集合 `movies` 建好：16 个点，状态 = green

green 是 Qdrant 的健康状态灯；也可以用 REST 亲自看：`curl http://localhost:6333/collections/movies`

STEP 4 高光时刻：语义搜索 vs 关键词搜索

怎么操作

运行 `python exp2_search.py` (内含 3 个"一个字都对不上"的查询)

目的

亲眼见证第 3 问的"四代进化"：字面匹配 0 命中，语义搜索全命中

实现了什么

理解 embedding 的价值——它检索的是"意思"而不是"字符串"

怎么验证

对比两组结果：关键词 0 条；语义 Top1 全部正确

✅ 预期结果 (实测输出)

查询「机器有了自我意识，人类活在虚假世界里」→ 关键词 0 条；语义 Top1 黑客帝国 0.67，Top2 流浪地球2 0.52，Top3 星际穿越 0.50

查询「在星空深处寻找人类未来的希望」→ 关键词 0 条；语义 Top1 星际穿越 0.63

查询「一个傻乎乎但心地善良的人的一生」→ 关键词 0 条；语义 Top1 阿甘正传 0.46

试着改 QUERIES 里的查询句子再跑几次，观察 score 排序如何变化——这是理解"语义空间"最直接的体感。

STEP 5 过滤搜索：语义 × 条件组合拳

怎么操作

运行 `python exp3_filter.py`

目的

体会第 9 问的“先滤后搜”：同一查询在不同 payload 条件下结果集如何收窄

实现了什么

掌握 `Filter + FieldCondition + MatchValue / range` 的标准写法

怎么验证

三次输出的片单应逐渐收窄且类型正确

✅ **预期结果**（实测输出，查询「关于动物的搞笑冒险」）

不过滤 Top3：疯狂动物城 0.48 / 千与千寻 0.40 / 盗梦空间 0.37

加 `genre=动画`：盗梦空间消失，换上你的名字 0.33

加 `genre=科幻 且 year≥2015`：只剩流浪地球2 0.36

STEP 6 收尾：直接用 REST API 玩数据库 + 优雅关闭

怎么操作

PowerShell:

`Invoke-RestMethod`

`http://localhost:6333/collections/movies |`

`ConvertTo-Json -Depth 5`

目的

记住 Qdrant 本质是 HTTP 服务——不写 Python 也能操作它（任何语言、甚至浏览器都能）

实现了什么

看懂集合的元信息：维度 512、距离 Cosine、点数 16

怎么验证

返回 JSON 里 `config.params.vectors.size = 512`

✅ **预期结果**（实测）返回含 `"points_count": 16`、`"status": "green"` 的 JSON。玩完后在原窗口

Ctrl+C 停掉 qdrant.exe 即可，数据已持久化，下次启动原样恢复。

注：Docker 版 Qdrant 自带 Web 控制台（/dashboard），官方 Windows 二进制不打包该前端（启动日志有 WARN），属正常差异，不影响任何功能。

坑与结论速查

Clash 代理劫持 localhost	qdrant-client 连本机报 502 Bad Gateway	<code>setx NO_PROXY "localhost,127.0.0.1"</code> (本机已设)
中文 Windows GBK	Python 脚本中文乱码/报错	<code>setx PYTHONUTF8 1</code> (本机已设)
ollama 拉模型失败	redirect target ... non-public 198.18.x.x	Clash fake-IP 劫持 DNS 所致; 改用 fastembed 或换网络环境
Windows 二进制无 dashboard	/dashboard 返回 404	官方二进制不打包 Web UI, Docker 版才有; 用 REST API 替代
Git Bash curl 返回空	命令"成功"但无输出	代理环境变量干扰; 验证改用 PowerShell 或 Python

Qdrant 向量数据库小白指南 · vector_database_qdrant 项目 · 2026-10-01 · 配套实验脚本在 exercise\ 目录 (exp1_ingest / exp2_search / exp3_filter)